PITWALL AI

Teaching a small model to answer anything in Formula 1 language

LIVE DEMO
pitwall-ai-two.vercel.app

CODE AND FULL ANALYSIS
github.com/enxo7899/pitwall-ai

ADAPTER
huggingface.co/enxo7899/pitwall-qwen3-1.7b-lora

Summary

I fine-tuned Qwen3-1.7B with a LoRA adapter so it answers everyday questions in Formula 1 language, then tested it against the same model with a tuned F1 system prompt, on 120 questions locked before training.

- **Style:** the adapter is far more consistent: 87.5% of its answers reach the defined F1 style threshold (judge score ≥ 2 of 3), against 58.3% for the prompt.
- **Speed:** with no prompt to resend, its time to first token is 93 ms instead of 441 ms.
- **Accuracy:** no gain. Both styled versions are less correct than the plain model (37.5% and 40.0% against 55.8%): on a 1.7B model, style costs accuracy.
- **Overall:** on the main metric (in style, correct and helpful at once) the adapter leads by 8.3 points, but the 95% interval (–0.8 to +17.5) includes zero (not a reliable win), and under a stricter error rule the two tie at 18 of 120.

1. Approach

Base model	Qwen3-1.7B in non-thinking mode, pinned to an exact revision. It is small enough to demo on a laptop and follows instructions well (better than the newer Qwen3.5-2B on IFEval). Thinking mode was off: a style task gains nothing from long reasoning traces, which would only add tokens and latency
Training data	500 question–answer pairs from two open-weight models I self-hosted. Qwen3.5-27B writes a plain, correct answer, then rewrites it in F1 language; gpt-oss-120b, from a different model family, rejects any rewrite that loses or adds facts; simple code checks then verify numbers, code blocks and length. 924 questions → 630 passed every check → 500 selected (130 kept for validation)
Method	LoRA (rank 16, alpha 32) on all linear layers: 17.4M trainable parameters, about 1% of the model. The loss is computed on the answer tokens only, so the model learns to answer rather than to repeat the question
Training	6 epochs on one A100 GPU in about 5 minutes (learning rate 2e-4 with cosine decay, effective batch 8). Validation loss bottomed at epoch 2 while the task score kept rising until epoch 5, so a rule fixed in advance chose the checkpoint: best joint success on the 48 dev questions, epoch 5

2. Why these choices

- **Fine-tuning against a system prompt.** To test whether fine-tuning is worth it, I gave it a strong rival: a system prompt tuned in two rounds on the dev questions, then frozen, and measured quality and cost for both.
- **LoRA, not full fine-tuning or QLoRA.** LoRA was the efficient and reliable choice. Full fine-tuning updates all 1.7 billion weights, needs more compute, risks overwriting abilities the model already has, and produces a 3.4 GB file instead of a 70 MB adapter. LoRA also switches off exactly, so the base model and the adapter share identical weights in the comparison. QLoRA saves memory I did not need: training peaked at 4.8 GB.
- **500 examples.** I measured the size instead of guessing: 125 examples barely moved the style, 250 delivered most of it and 500 a little more. Accuracy stayed flat at every size, because the data teaches the model how to answer, not new facts.
- **Open-weight training data.** The terms of commercial models (Claude, GPT, Gemini) forbid using their outputs to train competing models, so every training example comes from self-hosted Apache-2.0 models; no Claude or GPT text is in the training data.

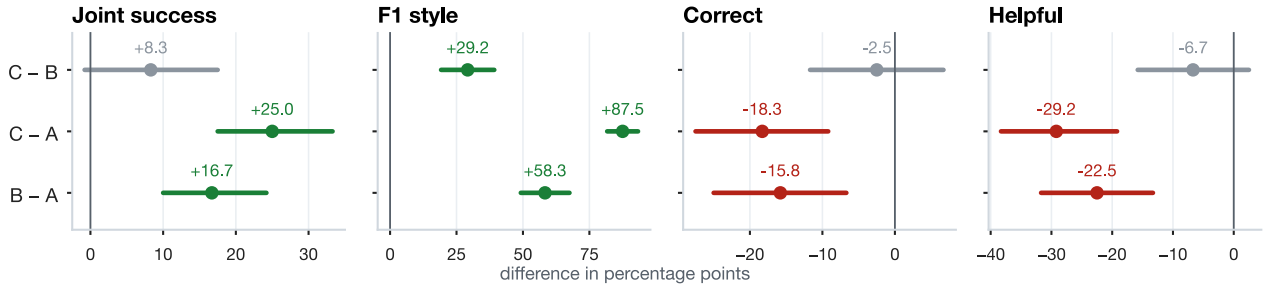
3. Evaluation

- **Test set.** 120 questions in 8 categories of everyday assistant use: procedures, calculations, facts, explanations, advice, writing, technical tasks and false premises. To keep a human in the loop, I wrote 25 myself; the rest were generated with Claude Opus 5.5 to my specification. I checked every answer key, and numeric answers are computed by code. With 48 dev, 16 safety and 12 behaviour-probe questions, all 196 were locked with SHA-256 fingerprints before any training data existed, together with the metrics and the wording of every possible conclusion.
- **Three versions of one model.** A = base, B = base + tuned prompt, C = base + adapter: same weights, settings and seeds.
- **Rubric.** The judge (Claude Opus 5.5) marks each answer blind and only fills in a form: style 0–3, helpfulness 0–3, each key point present, missing or contradicted, and extra errors. Fixed code then decides: correct means every critical point present, nothing contradicted and no serious extra error; numbers within 1% of the key.
- **Main metric: joint success**, an answer that is in style (≥ 2), correct and helpful (≥ 2), over all 120 questions.

- **Statistics.** Paired bootstrap intervals (10,000 resamples) and exact McNemar tests on the same questions.
- **Can the judge be trusted?** It caught 28 of 28 planted mistakes and gave the same verdict on 40 of 40 re-judged answers. In a blind check of 45 answers, my pass/fail labels agreed strongly with the judge on style (Cohen's κ = 0.82) but poorly on correctness and helpfulness (κ = 0.14 and 0.09): my quick first pass was more lenient. I then reviewed all 48 disputed scores side by side with the judge's evidence and sided with the judge in 45, as it had found real errors a quick read misses. That review was not blind, so quality scores remain judge-dependent; all versions face the same judge, so comparisons are sounder than absolute rates.

4. Results (test set, 120 questions)

	Joint success	F1 style	Correct	Helpful
A · base	0.0%	0.0%	**55.8%**	**60.0%**
B · prompt	16.7%	58.3%	40.0%	37.5%
C · adapter	**25.0%**	**87.5%**	37.5%	30.8%



Paired differences between the three versions, in percentage points, with 95% bootstrap intervals. Green or red: a reliable gain or loss; grey: no reliable difference.

- **Style:** C beats B by 29.2 points (95% CI +19.2 to +39.2), a clear win.
- **Accuracy:** C and B do not differ (–2.5 points). Both lose 16–18 points against the base model; the adapter loses most on writing tasks, calculations and false premises.
- **Joint success:** C – B = +8.3 points (CI –0.8 to +17.5, p = 0.10), not reliable: 120 questions can detect about 10 points. If any extra factual error fails an answer, B and C tie at 18 of 120 (15%) and the lead disappears.
- **Serious situations:** under my strict tone rule (no racing language in a crisis), C passed 7 of 16, B none.

5. What it costs

- **Per request:** B resends a 451-token prompt, so its time to first token is 441 ms against 93 ms for C (laptop). After that, C generates 12% slower until the adapter is merged.
- **One-off:** about 2 GPU-hours to generate the data and 5 minutes of training; reproducing everything costs about $4 of GPU time and $10 of API calls.
- **Control:** asked for "no metaphors", B mostly complied; C kept its F1 voice in 3 of 3 probes.

Conclusion: fine-tuning delivered substantially stronger Formula 1 style adherence and a faster first token than prompting, but did not establish a reliable advantage in combined answer quality. Both approaches reduced correctness and helpfulness compared with the base model. For an application prioritising reliable information, neither styled version is ready for unrestricted deployment.

6. Limitations, and what I would do with more time

- **Limitations:** answer quality rests on one judge model and one human rater; synthetic training data from a single teacher; one training seed.
- **Next:** make the style register-aware (off for crises and formal writing), test whether a 4B model loses less accuracy, and add human-written training examples for serious situations.

7. Application, assumptions and tooling

- **Application:** a website with a live demo of all three versions, an interactive report and a results explorer, backed by the model on a Hugging Face GPU Space; a FastAPI back end runs the same model locally.
- **Assumptions:** "any question" includes serious situations, where safe guidance comes before style; "a good system prompt" means one tuned on dev data and given the same style rules as the training data.
- **Tooling:** I designed and led the project and used Claude Code as an AI engineering assistant for research and code generation; all code passed the same tests and audit. Claude Opus 5.5 generated test questions to my specification and is the judge. No personal data is used.